

DEVOPS MASTERY

Batch 3

60-Day Live Training Program

From Zero to Production-Ready Engineer

[Stepaheadexcellence.com](https://stepaheadexcellence.com) | +91 7899151988

1.5 Hours Daily Live Sessions | 5 Hands-On Labs Per Module |

Linux • Docker • Kubernetes • Terraform • Git & GitHub • AWS • Azure • Jenkins • Grafana

Why This Course Will Transform Your Career

The world does not need more developers who simply write code. It needs engineers who understand how software breathes in production — how it is built, tested, secured, deployed, monitored, and scaled across thousands of nodes without breaking a sweat. That is exactly what this DevOps Mastery Program is designed to produce.

Over 60 days of live, instructor-led sessions of 90 minutes each, you will move from foundational concepts to production-grade engineering skills. Every single module is built around real-world scenarios drawn from companies like Netflix, Amazon, Spotify, and Uber — the organizations that invented and refined the DevOps culture. You will not be watching theory slides. You will be building things.

DevOps is currently the most in-demand skill set in the technology industry. According to industry surveys, organizations that adopt DevOps practices deploy code 200 times more frequently, recover from incidents 24 times faster, and experience 50% fewer failures. Engineers who understand this ecosystem command salaries ranging from 06 to 40 lakhs per annum in India and 90,000 to 160,000 USD globally.

This program covers every tool in the modern DevOps toolchain: Linux fundamentals, container technology with Docker, orchestration with Kubernetes, infrastructure as code with Terraform, version control with Git and GitHub, cloud platforms AWS and Azure, continuous integration with Jenkins, observability with Grafana, and the increasingly critical discipline of DevSecOps — security baked into every stage of the pipeline.

Every module includes 5 hands-on labs. Every lab builds something real. Every concept is explained with the depth of an engineer who has run these systems in production.

Module 1: DevOps Foundations and Culture

Before you touch a single tool, you need to understand why DevOps exists and what problem it solves. This module gives you that context — the history, the philosophy, and the technical vocabulary that will make every subsequent module click into place. Engineers who skip this foundation often find themselves using tools without understanding why they were invented, which leads to shallow knowledge that falls apart under pressure.

DevOps emerged from a very specific pain point: the wall between development teams and operations teams. Developers wanted to ship fast. Operations teams wanted stability. These goals were in constant conflict, and software deployments became slow, painful, fear-inducing events that happened once a quarter with all hands on deck. DevOps was not invented to solve a technical problem. It was invented to solve a cultural and organizational problem.

You will learn the CALMS framework — Culture, Automation, Lean, Measurement, and Sharing — which is the intellectual backbone of DevOps. You will understand the Three Ways: flow, feedback, and continuous learning. You will see how companies like Amazon went from quarterly releases to deploying code more than 10,000 times per day by applying these principles relentlessly.

The CI/CD pipeline model is introduced here at a conceptual level, allowing you to understand how code travels from a developer's laptop to a production server and what can go wrong at every step. By the end of Day 5, you will understand what DevOps is, what it is not, how it differs from Agile and SRE, and what your role as a DevOps engineer actually looks like day to day.

Daily Breakdown

Day 1: What is DevOps — History, Culture, and the CALMS Framework

The origin story of DevOps, from the Agile manifesto to Patrick Debois coining the term at a conference in Belgium in 2009. The wall between dev and ops explained with real case studies. The CALMS framework dissected with examples from Netflix, Google, and Etsy.

Day 2: Software Delivery Lifecycle and the Three Ways

How software actually moves from idea to production. The waterfall model and why it failed. Agile and Scrum as precursors. The Three Ways of DevOps explained in depth with concrete examples from teams that transformed their delivery processes.

Day 3: CI/CD Concepts and Pipeline Mental Model

What continuous integration, continuous delivery, and continuous deployment actually mean and how they differ. Drawing a pipeline on paper before ever writing code. Understanding build, test, deploy, and monitor stages. Introduction to pipeline tooling concepts.

Day 4: Site Reliability Engineering and DevSecOps Introduction

SRE as DevOps with an SLA. Error budgets, SLIs, SLOs, and SLAs explained with Google's original framework. Introduction to DevSecOps — the shift-left security model and why baking security into the pipeline is now a business requirement, not an afterthought.

Day 5: DevOps Toolchain Overview and Career Pathways

A complete walkthrough of the modern DevOps toolchain: source control, build tools, testing frameworks, container platforms, orchestrators, IaC tools, monitoring stacks. Career paths for DevOps engineers, platform engineers, SREs, and cloud architects.

Hands-On Labs — Module 1

Hands-On 1: Draw and document a complete software delivery pipeline for a sample e-commerce application, mapping each stage to a tool category and identifying potential failure points.

Hands-On 2: Research and present a case study on how one of Netflix, Amazon, or Google implemented DevOps culture, including the metrics that improved and the organizational changes they made.

Hands-On 3: Set up your DevOps learning environment: install WSL2 or configure a Linux VM, install VS Code with essential extensions, and configure your terminal with a productive shell profile.

Hands-On 4: Create a personal DevOps learning journal and document your understanding of the Three Ways with a real example from your own experience or observation.

Hands-On 5: Build a simple 'hello world' CI/CD concept map using a free visual tool, showing code commit, automated tests, staging deployment, and production release with rollback capability.



Module 2: Linux for DevOps Engineers

Linux is not just a prerequisite. It is the foundation upon which virtually all of modern cloud infrastructure runs. Every Docker container, every Kubernetes node, every cloud virtual machine you will ever work with runs on Linux. If you do not understand Linux deeply, you will always be working with borrowed knowledge, struggling to debug problems, and dependent on others when things go wrong.

This module does not teach you Linux like a textbook. It teaches you Linux the way a DevOps engineer uses it — from the command line, under pressure, with real systems to configure and real problems to solve. You will learn the filesystem hierarchy and understand why `/etc`, `/var`, `/usr`, and `/proc` exist and what lives in each. You will learn process management because you need to know how to find what is consuming your CPU at 2 AM when a production server is on fire.

Networking is given substantial attention in this module because a DevOps engineer who cannot diagnose a networking problem is severely limited. You will learn how to trace packets, configure firewalls, read socket states, and understand why your application cannot reach its database even when both are running.

Shell scripting receives deep coverage because automation in DevOps begins at the shell. Before you ever write a Terraform configuration or a Kubernetes manifest, you will write shell scripts that provision environments, check health endpoints, rotate logs, and notify you when something breaks. Shell scripting is the duct tape that holds pipelines together, and you will master it.

The module closes with coverage of `systemd`, cron jobs, and Linux security concepts including file permissions, `sudoers`, SSH hardening, and SELinux/AppArmor basics — all concepts that appear repeatedly in real DevOps work.

Daily Breakdown

Day 6: Linux Architecture, Filesystem Hierarchy, and Essential Commands

The Linux kernel and how it relates to user space. The Filesystem Hierarchy Standard explained in full. Navigation, file manipulation, text processing with cat, less, head, tail, grep, awk, sed, and cut. Understanding inodes, hard links, and symbolic links.

Day 7: Users, Groups, Permissions, and Access Control

The Unix permission model in depth: read, write, execute for owner, group, and others. chmod, chown, chgrp with real examples. SUID, SGID, and sticky bits. Introduction to Access Control Lists. Creating and managing users and groups. The sudoers file and secure privilege escalation.

Day 8: Processes, System Monitoring, and Performance Tuning

Process lifecycle: fork, exec, wait, exit. Foreground and background processes. The process tree and orphan processes. top, htop, ps, lsof, strace, vmstat, iostat, and sar for system analysis. Load average explained. Memory management: free, /proc/meminfo, and swap.

Day 9: Networking Fundamentals for DevOps

The TCP/IP stack from a DevOps perspective. ip, ss, netstat, tcpdump, dig, nslookup, curl, wget. Configuring static IPs and DNS. Firewall management with iptables and firewalld. SSH: key-based authentication, config files, port forwarding, and tunneling. Network namespaces as the foundation of container networking.

Day 10: Package Management, Software Installation, and Repositories

APT and YUM/DNF package managers in depth. Adding repositories, managing GPG keys, pinning versions. Building from source when packages are not available. Managing system services with systemd: start, stop, enable, disable, reload, restart. Journald and log management.

Day 11: Shell Scripting Part 1 — Variables, Control Flow, and Functions

Bash scripting from the ground up: shebang lines, variables, environment variables, positional parameters. Conditionals with if/elif/else and case statements. Loops: for, while, until with practical examples. Functions and scope. Exit codes and error handling with set -e and trap.

Day 12: Shell Scripting Part 2 — Advanced Patterns and Real-World Scripts

String manipulation, arrays, associative arrays. Regular expressions in bash. Process substitution and command substitution. Here documents. Writing production-grade scripts: logging, argument parsing, configuration file reading, and idempotent execution. Script testing and debugging techniques.

Day 13: File Systems, Disk Management, and Storage

Disk partitioning with fdisk and parted. Creating and mounting filesystems: ext4, xfs, btrfs. LVM: physical volumes, volume groups, logical volumes. NFS and CIFS/SMB mounts. /etc/fstab in depth. Disk usage analysis with df, du, and ncd. Understanding RAID concepts.

Day 14: Cron Jobs, Task Scheduling, and Automation

The cron daemon and crontab syntax in full detail. At and batch for one-time scheduling. Systemd timers as the modern cron alternative. Writing cron jobs that handle errors gracefully and send alerts. Anacron for systems that are not always on. Common automation patterns in DevOps using cron.

Day 15: Linux Security Hardening and SSH Best Practices

The CIA triad applied to Linux systems. SSH hardening: disabling root login, changing default ports, configuring AllowUsers, setting up fail2ban. File integrity monitoring with AIDE. Understanding SELinux and AppArmor contexts. Audit logging with auditd. Linux namespaces and cgroups as security boundaries — the foundations of containers.

Hands-On Labs — Module 2

Hands-On 1: Configure a fresh Ubuntu 22.04 server from scratch: create administrative users, harden SSH, configure a firewall with ufw, set up log rotation, and document every step as a runbook that another engineer could follow.

Hands-On 2: Write a comprehensive system health check script that reports CPU usage, memory usage, disk usage, top processes, open network connections, and failed systemd services — and sends an alert if any threshold is breached.

Hands-On 3: Simulate a production incident: a web server process is consuming 100% CPU, disk is 95% full, and a network port is blocked. Use Linux tools to diagnose and resolve each issue within a time limit.

Hands-On 4: Build an automated log analysis script that parses an Nginx access log, extracts the top 10 IP addresses by request count, identifies 5xx errors, and generates a daily HTML report.

Hands-On 5: Set up an LVM-based storage configuration, create a scheduled backup script using rsync and cron, implement log rotation, and test disaster recovery by restoring from the backup on a second machine.



Module 3: Docker and Containerization

Containers changed software engineering more profoundly than any technology since virtualization. Before containers, deploying an application meant meticulously configuring a server, managing software dependencies across environments, and dealing with the infamous 'it works on my machine' problem. Containers solve this by packaging an application with everything it needs — the runtime, libraries, configuration, and code — into a single, portable unit that runs identically everywhere.

Docker did not invent containerization — Linux namespaces and cgroups had existed for years — but it made containers accessible to every developer and operator by wrapping complex kernel primitives in a simple, elegant interface. Today, Docker is the lingua franca of container technology, and understanding it deeply is non-negotiable for any DevOps engineer.

This module begins with the conceptual underpinning: what is a container, how does it differ from a virtual machine, and why does that difference matter? You will learn about Linux namespaces — the mechanism that gives each container its own isolated view of the filesystem, network, and process tree — and cgroups, which limit how much CPU and memory a container can consume. Understanding these foundations helps you debug container problems that would otherwise be opaque.

You will learn to write Dockerfiles that produce small, secure, production-ready images. This is a skill that separates junior practitioners from senior ones: most people can write a Dockerfile that works; very few write Dockerfiles that are optimized, secure, and maintainable. You will learn multi-stage builds, layer caching strategies, non-root user configurations, and image scanning for vulnerabilities — all DevSecOps practices that are increasingly required in enterprise environments.

Docker Compose receives substantial coverage because it is the tool most teams use to run multi-container applications locally and in staging environments. By the end of this module, you will understand Docker networking, Docker volumes, container registries, and the principles that make containers a fundamental primitive in modern infrastructure.

Daily Breakdown

Day 16: Container Fundamentals — Namespaces, cgroups, and the Container Model

What a container actually is at the kernel level. Linux namespaces: pid, net, mnt, uts, ipc, user. How cgroups limit resource consumption. Comparing containers and virtual machines: isolation model, overhead, startup time, and use cases. Docker architecture: daemon, client, and registry. Installing Docker and running your first containers.

Day 17: Docker Images — Layers, Dockerfiles, and Build Optimization

How Docker images are constructed as layered filesystems using UnionFS. Writing Dockerfiles: FROM, RUN, COPY, ADD, ENV, ARG, WORKDIR, EXPOSE, CMD, ENTRYPOINT explained in depth with trade-offs. Layer caching and how to structure Dockerfiles to maximize cache hits. Multi-stage builds for producing minimal production images.

Day 18: Docker Volumes, Storage Drivers, and Data Persistence

The container filesystem is ephemeral: everything dies when the container stops. Volumes: named volumes, bind mounts, and tmpfs explained with use cases. Volume drivers for network storage. The difference between COPY and volume mounts. Backing up and restoring Docker volumes. Choosing the right storage driver for your workload.

Day 19: Docker Networking — Bridge, Host, Overlay, and Custom Networks

Docker's networking model: the bridge network and why containers get private IPs. How containers communicate with each other and the outside world. Custom bridge networks and service discovery by container name. Host networking for performance-critical workloads. Macvlan and IPvlan for advanced network configurations. Network policies and security considerations.

Day 20: Docker Compose for Multi-Container Applications

The docker-compose.yml format in full: services, networks, volumes, environment variables, and dependency management with depends_on and healthchecks. Running databases, caches, message queues, and application servers together. Override files for environment-specific configurations. The compose watch feature for development workflows.

Day 21: Container Registries, Image Security, and DockerHub to ECR

Public and private container registries: Docker Hub, Amazon ECR, GitHub Container Registry, Google Artifact Registry. Tagging strategies: semantic versioning, commit SHA, latest tag considerations. Image scanning with Trivy and Docker Scout. Base image selection for security: scratch, distroless, Alpine, and Debian Slim trade-offs. Signing images with Docker Content Trust.

Day 22: Docker in Production — Best Practices, Logging, and Monitoring

Running containers in production requires understanding resource limits, restart policies, health checks, and log management. Configure CPU and memory limits to prevent a single container from starving others. Centralized logging with the json-file driver and forwarding to ELK or CloudWatch. Container monitoring with Prometheus and cAdvisor preview. Introduction to container runtime security with Falco.

Hands-On Labs — Module 3

Hands-On 1: Containerize a three-tier web application (React frontend, Node.js API, PostgreSQL database) from scratch, write optimized multi-stage Dockerfiles for each component, reduce final image sizes using distroless base images, and verify the application runs correctly end-to-end.

Hands-On 2: Write a Docker Compose configuration for a complete local development environment including the application, a PostgreSQL database with persistent volumes, Redis cache, Nginx reverse proxy, and a mail testing server. Implement health checks for all services.

Hands-On 3: Perform a full image security audit: scan a set of provided images with Trivy, identify and remediate critical CVEs by updating base images and packages, document the before and after vulnerability counts, and write a Dockerfile that runs as a non-root user.

Hands-On 4: Build a private container registry with Docker Registry or Harbor, push custom images to it, configure pull secrets, set up image retention policies, and implement a workflow where a new image is automatically pushed after a successful build script runs.

Hands-On 5: Simulate a production debugging scenario: a containerized application is failing silently. Use docker logs, docker exec, docker inspect, docker stats, and network diagnostic tools to identify that the container is out of memory, cannot reach its database due to a network misconfiguration, and has a misconfigured health check endpoint. Fix all three issues.

Module 4: Kubernetes and Container Orchestration

If Docker is the shipping container, Kubernetes is the entire port. It is the platform that decides where containers run, restarts them when they fail, scales them when traffic increases, and routes network traffic to healthy instances. Kubernetes — often abbreviated as K8s — has become the de facto standard for running containerized workloads at scale, adopted by organizations from small startups to the world's largest technology companies.

Understanding Kubernetes requires mental model shifts. You must stop thinking about individual servers and start thinking about a cluster — a pool of compute resources that you describe declaratively. You do not tell Kubernetes to 'run this container on that server.' You tell Kubernetes 'I want three replicas of this application running at all times, using no more than 500 megabytes of memory each, accessible on port 8080.' Kubernetes figures out the rest, and it keeps your intent satisfied even as nodes fail and pods crash and node capacity fluctuates.

This module covers the full Kubernetes architecture: the control plane components (API Server, etcd, Scheduler, Controller Manager), the worker node components (kubelet, kube-proxy, container runtime), and how they work together to maintain desired state. You will learn every core workload resource: Pods, ReplicaSets, Deployments, StatefulSets, DaemonSets, and Jobs.

Service discovery, networking, and ingress receive deep attention because they are among the most common sources of confusion for new Kubernetes practitioners. You will also learn storage with PersistentVolumes and PersistentVolumeClaims, configuration management with ConfigMaps and Secrets, and horizontal and vertical pod autoscaling.

The module incorporates recent trends including Helm for package management, Kustomize for configuration customization, the Gateway API as the successor to Ingress, and an introduction to service meshes with Istio or Linkerd. Security coverage includes RBAC, Pod Security Admission, Network Policies, and image policy webhooks.

Daily Breakdown

Day 23: Kubernetes Architecture — Control Plane, Worker Nodes, and etcd

The complete Kubernetes architecture explained component by component. The API Server as the front door of the cluster. etcd as the cluster's brain: what it stores, how it works, and why you should back it up. The Scheduler's decision-making process. Controller Manager and the reconciliation loop — the heart of Kubernetes philosophy. The kubelet and how it talks to the container runtime. Setting up a local cluster with minikube or kind.

Day 24: Pods, ReplicaSets, and Deployments

The Pod as the fundamental unit of Kubernetes: why it wraps one or more containers together. Pod lifecycle states: Pending, Running, Succeeded, Failed, Unknown. ReplicaSets and how they ensure availability. Deployments as the standard way to manage stateless applications: rolling updates, rollback, pause and resume. Deployment strategies: Recreate vs RollingUpdate with maxSurge and maxUnavailable.

Day 25: Services, DNS, and Internal Networking

How Pods get IP addresses and why those IPs are ephemeral. Services as stable network endpoints: ClusterIP, NodePort, LoadBalancer, and ExternalName types explained with use cases. kube-proxy and how it implements service routing. CoreDNS and Kubernetes service discovery by name. Headless services for stateful applications. EndpointSlices and the future of service routing.

Day 26: Ingress, Gateway API, and External Traffic Management

Exposing applications to the outside world: the evolution from NodePort to LoadBalancer to Ingress. The Ingress resource and Ingress controllers: Nginx, Traefik, and HAProxy. TLS termination and certificate management with cert-manager. The Kubernetes Gateway API as the modern successor to Ingress: HTTPRoute, GRPCRoute, and TCPRoute. Configuring external DNS for automatic DNS management.

Day 27: ConfigMaps, Secrets, and Configuration Management

The problem of configuration: how do you change application behavior without rebuilding images? ConfigMaps for non-sensitive configuration: mounting as volumes vs injecting as environment variables. Secrets for sensitive data: Base64 encoding is not encryption. Sealed Secrets and External

Secrets Operator for GitOps-safe secret management. HashiCorp Vault integration for enterprise secret management.

Day 28: Persistent Storage — PVs, PVCs, and Storage Classes

Why stateful applications are harder in Kubernetes. PersistentVolumes and PersistentVolumeClaims: the abstraction that separates storage provisioning from consumption. StorageClasses and dynamic provisioning. Access modes: ReadWriteOnce, ReadWriteMany, ReadOnlyMany. StatefulSets for databases and other stateful workloads: stable network identities and ordered deployment. Volume snapshots and backup strategies.

Day 29: RBAC, Security Policies, and Kubernetes Hardening

Kubernetes security is layered. RBAC: Roles, ClusterRoles, RoleBindings, ClusterRoleBindings with practical examples. ServiceAccounts and the principle of least privilege. Pod Security Admission replacing the deprecated PodSecurityPolicy. Network Policies for microsegmentation: deny-all by default with explicit allow rules. Image admission webhooks. Auditing and runtime security with Falco. CIS Kubernetes Benchmark overview.

Day 30: Helm, Kustomize, and Application Package Management

Managing Kubernetes manifests at scale is complex. Helm: charts, values.yaml, templates, releases, and repositories. Writing your own Helm chart from scratch. Helm hooks for pre-install and post-upgrade tasks. Kustomize: overlays, patches, and strategic merge patches as an alternative to Helm templating. When to use Helm vs Kustomize vs raw manifests. ArgoCD and Flux as GitOps deployment tools.

Day 31: Horizontal Pod Autoscaling, Vertical Pod Autoscaling, and Cluster Autoscaling

Kubernetes scales in three dimensions: pods horizontally, pods vertically, and nodes automatically. HPA: CPU and memory based scaling with custom metrics using KEDA. VPA: understanding resource requests and limits, and letting Kubernetes recommend optimal values. Cluster Autoscaler and Karpenter: scaling the node pool based on pending pod demand. Cost optimization strategies: spot instances, right-sizing, and bin packing.

Hands-On Labs — Module 4

Hands-On 1: Deploy a complete microservices application on Kubernetes with at least four services (frontend, backend API, database, cache). Write all manifests from scratch — no copy-paste. Implement Deployments, Services, ConfigMaps, Secrets, resource limits, and health probes for every component.

Hands-On 2: Implement a blue-green deployment strategy and a canary release strategy for a sample application using Ingress weight annotations or the Gateway API. Demonstrate zero-downtime deployments and the ability to rollback instantly when a deployment shows high error rates.

Hands-On 3: Configure a complete RBAC model for a multi-team cluster: create namespaces for team-alpha and team-beta, define Roles that allow developers to view and edit in their own namespace but not others, create admin ClusterRoles, implement Network Policies to isolate namespaces, and verify the controls work by attempting forbidden operations.

Hands-On 4: Package the microservices application from Lab 1 as a Helm chart with environment-specific values files for development, staging, and production. Add Helm hooks for database migrations, implement chart tests with helm test, and push the chart to a chart repository.

Hands-On 5: Configure Horizontal Pod Autoscaling for the backend API using CPU metrics and a custom KEDA trigger (HTTP request rate or queue depth). Load test the application with k6 or Locust, observe pods scaling up automatically, scale down after load subsides, and document the scaling behavior in a report.

Module 5: Terraform and Infrastructure as Code

Imagine having to provision 50 cloud servers, 20 load balancers, 10 databases, a dozen security groups, and a VPC with subnets spread across three availability zones — and then doing it again for staging and production. Doing this by clicking through a cloud console is not only tedious; it is error-prone, undocumented, and irreproducible. Infrastructure as Code solves this by letting you describe your infrastructure in code files that can be version controlled, reviewed, tested, and applied automatically.

Terraform, developed by HashiCorp and now maintained as an open-source project under the OpenTF Foundation (forked as OpenTofu), is the most widely adopted IaC tool in the industry. It uses a declarative configuration language called HCL — HashiCorp Configuration Language — to describe the desired state of your infrastructure. You write what you want, and Terraform figures out how to get there.

What makes Terraform powerful is its provider ecosystem. With hundreds of providers, Terraform can manage resources across every major cloud provider — AWS, Azure, GCP — as well as DNS providers, monitoring platforms, database services, and even Kubernetes clusters. You write one unified language to manage your entire infrastructure stack.

This module covers Terraform from first principles to advanced production patterns. You will learn the core workflow (init, plan, apply, destroy), the HCL language (variables, outputs, locals, data sources, functions), module design for reusability, and state management — including the critical topic of remote state with locking to enable team collaboration without conflict.

Advanced topics include Terraform workspaces, dynamic blocks, `for_each` and `count` meta-arguments, provider aliasing for multi-region deployments, and integrating Terraform into CI/CD pipelines with automated plan and apply stages. Security topics include secret injection via environment variables, Sentinel policies for compliance, and drift detection.

Daily Breakdown

Day 32: IaC Concepts and Terraform Architecture

The spectrum of IaC tools: imperative vs declarative, mutable vs immutable infrastructure. Why Terraform won the IaC wars. Terraform's architecture: providers, resources, data sources, and the state file. Installing Terraform and the OpenTofu alternative. Writing your first configuration: provisioning an EC2 instance or Azure VM from a blank file.

Day 33: HCL Language Deep Dive — Variables, Outputs, and Data Sources

HCL syntax: blocks, arguments, expressions, and references. Variable types: string, number, bool, list, map, set, object, tuple. Input variables with validation rules. Output values and their role in module composition. Data sources for querying existing infrastructure. Local values for DRY code. Built-in functions: string manipulation, numeric operations, collection transformations.

Day 34: Terraform State Management and Remote Backends

The state file is Terraform's memory: it maps your configuration to real infrastructure. Why local state files are dangerous in teams. Remote backends: S3 with DynamoDB locking for AWS, Azure Blob Storage, Terraform Cloud. State operations: list, show, mv, rm, import, and taint. State file security: encryption at rest, access control, and never storing secrets in state. Terraform drift detection and remediation.

Day 35: Modules — Building Reusable Infrastructure Components

Modules are the unit of reuse in Terraform. Building a module from scratch: inputs, outputs, and internal resources. Module composition: modules calling other modules. The Terraform Registry and using community modules responsibly. Module versioning with git tags. Testing modules with Terratest. Designing a module library for an organization: VPC module, compute module, database module, security module.

Day 36: Terraform Workspaces, Multi-Environment, and Multi-Region Patterns

Managing multiple environments (dev, staging, production) with Terraform. Workspaces: what they are, when to use them, and when not to. The monorepo vs polyrepo debate for Terraform code. Directory-based environment separation. Provider aliasing for multi-region and multi-account deployments. Using `for_each` and `count` to provision resources at scale. The Terragrunt wrapper for managing complex directory structures.

Day 37: Terraform in CI/CD Pipelines and GitOps for Infrastructure

Automating Terraform: running plan in pull requests and apply after merge. The atlantis, Spacelift, and Terraform Cloud approaches to automated Terraform. Handling secrets in automated pipelines: OIDC authentication with AWS and Azure without storing credentials. Policy as code with Sentinel and Open Policy Agent. The GitOps model for infrastructure: treating your cloud state as a repository.

Day 38: Advanced Terraform — Dynamic Blocks, Lifecycle Rules, and Provisioners

Dynamic blocks for conditionally creating nested resource attributes. The lifecycle meta-argument: `create_before_destroy`, `prevent_destroy`, and `ignore_changes`. Null resources and local-exec provisioners for escape hatches. The moved block for infrastructure refactoring without destroying resources. Import blocks for bringing existing infrastructure under Terraform management. Terraform graph visualization for dependency analysis.

Hands-On Labs — Module 5

Hands-On 1: Build a complete AWS VPC from scratch using Terraform: VPC, public and private subnets across 3 AZs, internet gateway, NAT gateways, route tables, security groups, and a bastion host. All resources tagged consistently and no hardcoded values anywhere in the code.

Hands-On 2: Create a reusable Terraform module library: a VPC module, an ECS or EC2 module, an RDS module, and an ALB module. Use these modules to provision a three-tier application infrastructure for both a staging and production environment using different variable files.

Hands-On 3: Implement remote state management with S3 and DynamoDB, migrate an existing local state file to remote backend, configure state locking, and simulate a conflict scenario where two engineers apply simultaneously to verify locking works.

Hands-On 4: Integrate Terraform into a GitHub Actions pipeline: run terraform fmt and validate on every push, post a formatted terraform plan as a pull request comment, require approval before terraform apply, and notify a Slack channel on successful apply or failure.

Hands-On 5: Write an Open Policy Agent (OPA) policy that enforces tagging standards, prevents provisioning of expensive instance types without an approval tag, blocks creation of public S3 buckets, and requires encrypted RDS instances. Test the policy against configurations that should pass and fail.

Module 6: Git, GitHub, and GitOps

Version control is the foundation of every engineering discipline. Without it, software development is guesswork — you cannot know what changed, when it changed, who changed it, or why. Git is the universal version control system, used by virtually every software project on earth, from individual side projects to the Linux kernel itself. GitHub is the platform that transformed Git into a collaborative development ecosystem, adding pull requests, code review, issue tracking, project management, and now CI/CD automation through GitHub Actions.

Many engineers think they know Git because they know `git add`, `commit`, and `push`. This module will show you that Git is a far richer tool. You will understand the Git object model — commits, trees, blobs, and tags — and how this model enables the powerful operations that make Git the best version control system ever built. When you understand what a branch actually is (a pointer to a commit), merge strategies, rebasing, and cherry-picking become obvious and intuitive rather than mysterious.

Branching strategies are given substantial attention because a poor branching strategy can make an engineering team miserable. You will learn Gitflow, trunk-based development, and the GitHub flow model, and you will understand when each is appropriate. You will learn how to write commit messages that are informative, how to use interactive rebase to clean up your history before sharing it, and how to handle merge conflicts without losing work.

GitHub Actions is one of the most powerful and accessible CI/CD platforms available today, and it receives detailed coverage. You will build workflows that test, build, containerize, scan, and deploy applications automatically on every push. The module concludes with an introduction to GitOps — the practice of using Git as the single source of truth for both application code and infrastructure state — using tools like ArgoCD and Flux.

Daily Breakdown

Day 39: Git Internals — The Object Model and How Git Really Works

Git stores everything as objects: blobs for file content, trees for directories, commits for snapshots, and tags for named references. Understanding the object model makes every Git operation predictable. The .git directory dissected: objects, refs, HEAD, index. How staging area works. The commit graph and how branches, tags, and HEAD are just pointers.

Day 40: Branching, Merging, Rebasing, and Conflict Resolution

Creating and switching branches. Fast-forward vs three-way merges. Merge commits and why they can clutter history. Rebasing: linear history at the cost of rewriting commits. Interactive rebase for cleaning up feature branches. Cherry-picking specific commits. Resolving merge conflicts like an expert: understanding both sides, using vimdiff or VS Code merge editor. The golden rule of rebasing: never rebase shared branches.

Day 41: Branching Strategies — Gitflow, Trunk-Based, and GitHub Flow

The branching strategy shapes how teams collaborate. Gitflow: master, develop, feature, release, and hotfix branches — complex but powerful for release-based software. Trunk-based development: everyone commits to main with feature flags and short-lived feature branches — the strategy used by Google, Facebook, and modern high-velocity teams. GitHub Flow: a simplified model for teams deploying continuously. Choosing the right strategy for your team size and release frequency.

Day 42: GitHub — Pull Requests, Code Review, and Collaboration

Forks vs clones and when to use each. Writing pull requests that get reviewed and merged quickly: description templates, linking issues, small focused changes. Code review best practices: what to look for, how to give constructive feedback, how to request changes without demoralizing teammates. Branch protection rules, required reviewers, and status checks. GitHub Projects for issue tracking and sprint management.

Day 43: GitHub Actions — Building CI Workflows

GitHub Actions architecture: workflows, jobs, steps, and actions. YAML workflow syntax in full. Triggers: push, pull_request, schedule, workflow_dispatch, and workflow_call. Using marketplace actions vs writing custom actions. Building a CI pipeline: checkout, setup environment, install

dependencies, lint, unit test, integration test, build artifacts, publish test results. Caching dependencies for faster builds. Matrix builds for testing across multiple environments.

Day 44: GitHub Actions — CD Pipelines, Environments, and Secrets

Continuous delivery with GitHub Actions: deploying to staging automatically, requiring manual approval for production. Environment-specific secrets and variables. OIDC-based authentication with AWS and Azure — no more long-lived credentials in GitHub Secrets. Deployment strategies: rolling, blue-green, canary using GitHub Actions with Kubernetes. Reusable workflows for sharing pipeline logic across repositories.

Day 45: GitOps with ArgoCD and Flux

GitOps is the practice of using Git as the single source of truth for all system configuration. The pull-based model: an agent in the cluster pulls desired state from Git rather than a pipeline pushing changes in. ArgoCD: installation, application CRDs, sync policies, health status, and the ArgoCD UI. Flux: GitRepository, Kustomization, and HelmRelease resources. Multi-tenancy GitOps with isolated environments per team. Secrets management in GitOps workflows.

Hands-On Labs — Module 6

Hands-On 1: Simulate a complete team development workflow: three team members (using separate branches) work on features, create pull requests with proper descriptions, conduct cross-reviews, resolve an intentional merge conflict, and get code merged to main using a trunk-based development strategy with branch protection rules.

Hands-On 2: Build a complete GitHub Actions CI pipeline for a Node.js or Python application: linting with ESLint or flake8, unit tests with coverage reporting, dependency vulnerability scanning with npm audit or safety, Docker image build and push to GitHub Container Registry, image scanning with Trivy, and a pull request comment showing test coverage and vulnerability report.

Hands-On 3: Create a GitHub Actions CD workflow that deploys to a staging Kubernetes cluster automatically on merge to main, sends a Slack notification with deployment details, requires a manual approval gate for production, deploys to production with a canary strategy, and rolls back automatically if the error rate exceeds 5% in the first 5 minutes post-deploy.

Hands-On 4: Set up ArgoCD on a Kubernetes cluster, create an application that tracks a Git repository, implement the app-of-apps pattern for managing multiple services, demonstrate GitOps in action by pushing a change to Git and watching ArgoCD automatically sync the cluster, and test the out-of-sync detection when someone makes a manual kubectl change.

Hands-On 5: Create a custom GitHub Action that runs automated compliance checks on Terraform code: verifies required tags, checks for open security group rules, validates naming conventions, and posts a detailed report as a pull request check. Publish the action to a private organization marketplace.

Module 7: AWS for DevOps Engineers

Amazon Web Services is the world's largest cloud platform, hosting more than a third of all cloud workloads globally. For a DevOps engineer, AWS is not just a place to rent servers — it is a complete platform for building, deploying, monitoring, and scaling applications. The services you choose, how you configure them, and how they interact with each other determine whether your application is resilient, scalable, and cost-efficient, or fragile, slow, and expensive.

This module approaches AWS from a DevOps perspective rather than a generic cloud overview. You will learn the services that DevOps engineers interact with daily: compute with EC2 and ECS and EKS, networking with VPC and Route 53 and CloudFront, storage with S3 and EBS and EFS, managed databases with RDS and DynamoDB, container orchestration with EKS, and the extensive suite of AWS DevOps services — CodePipeline, CodeBuild, CodeDeploy, and CodeArtifact.

Security in AWS is treated as a first-class concern throughout. You will learn IAM in depth — not just how to create users and policies, but the difference between identity-based and resource-based policies, the permission boundary model, the principle of least privilege in practice, and the AWS Organizations and Service Control Policies that govern large enterprise accounts.

Cost management is a critical DevOps skill that is often neglected in training programs. This module covers AWS Cost Explorer, Trusted Advisor, right-sizing recommendations, Spot instances for cost-effective batch workloads, and Reserved Instance strategies for predictable production workloads.

Daily Breakdown

Day 46: AWS Fundamentals — Regions, AZs, IAM, and VPC Architecture

The global AWS infrastructure: regions, availability zones, edge locations, and Local Zones. IAM architecture in depth: users, groups, roles, policies, and permission boundaries. The assume-role model and cross-account access. VPC design for production: public subnets, private subnets, NAT gateways, VPC endpoints, Transit Gateway, and VPC peering. Security Groups vs Network ACLs.

Day 47: Compute — EC2, Auto Scaling, ECS, and Lambda

EC2 instance families and selecting the right instance type. Auto Scaling Groups: launch templates, scaling policies (target tracking, step, scheduled), and lifecycle hooks. Amazon ECS: task definitions, services, and clusters. The Fargate launch type for serverless containers. AWS Lambda: event-driven compute, function configuration, layers, and cold start mitigation. When to choose EC2 vs ECS vs Lambda vs EKS.

Day 48: Storage, Databases, and Managed Services

S3: bucket policies, versioning, lifecycle rules, replication, S3 Select, and event notifications. EBS: volume types (gp3, io2, sc1), snapshots, and encryption. EFS for shared filesystem use cases. RDS: Multi-AZ for high availability, Read Replicas for scale, automated backups, and parameter groups. Aurora Serverless for variable workloads. DynamoDB for high-throughput low-latency applications: partition key design, GSIs, and DynamoDB Streams.

Day 49: AWS DevOps Services — CodePipeline, CodeBuild, and CodeDeploy

AWS CodeCommit as a managed Git service. CodeBuild: buildspec.yml syntax, build environments, caching, and test reporting. CodeDeploy: deployment groups, deployment configurations, and lifecycle hooks for EC2 and ECS. CodePipeline: stages, actions, approvals, and cross-region pipelines. CodeArtifact for managing npm, Maven, and Python packages. Integrating GitHub with AWS CodePipeline via OIDC connections.

Day 50: Amazon EKS, CloudWatch, and AWS Security Services

Amazon EKS: managed Kubernetes control plane, worker node groups vs Fargate, add-ons (CoreDNS, kube-proxy, VPC CNI, EBS CSI driver). ALB Ingress Controller and AWS Load Balancer Controller. CloudWatch: metrics, logs, dashboards, alarms, and Container Insights for

EKS monitoring. AWS Security Hub for centralized security findings. GuardDuty for threat detection. AWS Config for compliance auditing. Macie for S3 data sensitivity scanning.

Hands-On Labs — Module 7

Hands-On 1: Design and provision a production-ready AWS VPC using Terraform: multi-AZ architecture, public and private subnets, NAT gateways, VPC endpoints for S3 and DynamoDB, bastion host with Session Manager access (no SSH key required), and flow logs to CloudWatch.

Hands-On 2: Deploy a containerized application on Amazon ECS with Fargate: write a task definition, create an ECS service, configure an Application Load Balancer, set up auto-scaling based on CPU utilization, enable CloudWatch Container Insights, and configure centralized logging to CloudWatch Logs.

Hands-On 3: Build a complete AWS CI/CD pipeline using CodePipeline: source from GitHub, build with CodeBuild (unit tests, Docker build, ECR push), deploy to ECS with CodeDeploy using blue-green deployment strategy, implement a manual approval stage before production, and configure SNS notifications for pipeline events.

Hands-On 4: Implement a comprehensive IAM security model for a multi-team environment: create IAM roles with permission boundaries for developers, operations, and security teams; configure AWS Organizations SCPs to prevent regions outside approved list; set up AWS Config rules for compliance; enable AWS Security Hub and remediate any critical findings.

Hands-On 5: Conduct a full AWS cost optimization exercise on a provided account: use Cost Explorer and Trusted Advisor to identify waste, right-size over-provisioned EC2 instances, convert On-Demand to Reserved Instances for steady-state workloads, configure S3 lifecycle policies to move old objects to Glacier, and produce a report showing projected monthly savings.

Module 8: Azure DevOps and Microsoft Cloud

Microsoft Azure is the second-largest cloud platform globally and is the dominant choice in enterprise environments, particularly those already using Microsoft 365, Active Directory, and the broader Microsoft ecosystem. For DevOps engineers, Azure offers a uniquely integrated toolchain through Azure DevOps — a platform that combines repository hosting, CI/CD pipelines, artifact management, test planning, and project management in a single suite.

This module approaches Azure from the perspective of an engineer migrating from on-premises infrastructure or an AWS-familiar engineer learning a second cloud platform. The architectural concepts are similar but the terminology, services, and specific configurations differ in important ways. You will learn Azure Resource Manager and the ARM/Bicep template model, Azure Active Directory and its role in identity management, and the Azure networking model with Virtual Networks, subnets, and Network Security Groups.

Azure Kubernetes Service receives dedicated attention because it is the most common destination for containerized workloads in enterprise Azure environments. Azure DevOps Pipelines — both classic and YAML-based — are covered in depth, along with the integration between Azure DevOps and GitHub that Microsoft has developed as GitHub became part of the Microsoft ecosystem.

Modern Azure features including Azure Container Apps, Azure Container Instances, and the Azure Developer CLI are introduced as productivity tools that complement the core platform services

Daily Breakdown

Day 51: Azure Fundamentals — Regions, Resource Groups, ARM, and Bicep

Azure global infrastructure: regions, availability zones, and availability sets. The Azure Resource Manager model: resource groups, subscriptions, and management groups. ARM templates vs Bicep: why Bicep is the modern choice. Azure CLI and PowerShell for automation. Azure Policy for governance: initiative definitions, assignment scopes, and compliance reporting. Azure Blueprints for repeatable environment setup.

Day 52: Azure Networking, Active Directory, and Identity

Azure Virtual Networks: subnets, Network Security Groups, Application Security Groups. Azure Bastion for secure RDP/SSH without public IPs. Azure Front Door, Application Gateway, and Load Balancer for traffic management. Azure Active Directory: tenants, users, groups, and service principals. Managed Identities: system-assigned vs user-assigned, and why they eliminate credential management. Azure Key Vault for secrets, keys, and certificates.

Day 53: Azure Kubernetes Service and Azure Container Apps

AKS architecture: managed control plane, node pools, and virtual node integration. Configuring AKS with Workload Identity instead of service principals. Azure CNI vs kubenet networking in AKS. Azure Monitor Container Insights for AKS observability. AGIC (Application Gateway Ingress Controller) for enterprise-grade ingress. Azure Container Apps: the serverless container platform built on Kubernetes without the complexity. When to use AKS vs Container Apps vs Container Instances.

Day 54: Azure DevOps Pipelines — YAML Pipelines, Environments, and Approvals

Azure DevOps organization and project structure. Repositories: Azure Repos vs GitHub integration. YAML pipeline syntax: stages, jobs, steps, templates, and parameters. Service connections for authenticating with cloud resources using workload identity federation. Pipeline environments with approval gates and deployment strategies. Variable groups and Azure Key Vault integration for secrets. Pipeline templates for organizational reuse.

Day 55: Azure Monitor, Log Analytics, and Security Center

Azure Monitor as the unified observability platform: metrics, logs, alerts, and dashboards. Log Analytics workspace: KQL (Kusto Query Language) for log analysis — a powerful skill that sets

Azure engineers apart. Application Insights for distributed application monitoring: traces, dependencies, exceptions, and custom metrics. Microsoft Defender for Cloud: secure score, security recommendations, and threat protection. Azure Sentinel as a cloud-native SIEM.

Hands-On Labs — Module 8

Hands-On 1: Provision a complete Azure landing zone using Bicep: management groups hierarchy, subscriptions, resource groups, Azure Policy assignments for tagging and allowed regions, Azure Monitor diagnostic settings, and a Log Analytics workspace with required solutions enabled.

Hands-On 2: Deploy an AKS cluster with enterprise configuration: Workload Identity, Azure CNI, managed add-ons, node auto-provisioning, Azure Monitor Container Insights, and a sample microservices application. Configure AGIC for HTTPS ingress with a certificate from Azure Key Vault.

Hands-On 3: Build a multi-stage Azure DevOps YAML pipeline: build and test a .NET or Node.js application, build and push a Docker image to Azure Container Registry, deploy to AKS dev environment automatically, require approval for staging, run smoke tests, require a second approval for production, with full Slack notifications.

Hands-On 4: Implement Azure security best practices: configure Defender for Cloud, remediate the top 5 security recommendations, write KQL queries to detect suspicious login attempts, create alerts for critical security events, and configure Microsoft Sentinel with a custom analytics rule.

Hands-On 5: Migrate a simulated on-premises application to Azure using the strangler fig pattern: use Azure API Management as a facade, gradually route traffic to new Azure-hosted services, configure Azure Front Door for global routing, implement A/B testing at the routing layer, and monitor both old and new services side-by-side in Application Insights.

Module 9: Jenkins and Advanced CI/CD

Jenkins is the original open-source CI/CD server, first released in 2005 as Hudson and renamed in 2011. Despite its age, Jenkins remains the most widely deployed CI/CD tool in enterprise environments, primarily because of its flexibility, its massive plugin ecosystem of over 1,800 plugins, and the fact that it runs on-premises, which many regulated industries and enterprise security teams require.

Jenkins is famously complex. Setting it up correctly, keeping it secure, and building maintainable pipelines requires knowledge that this module delivers in concentrated form. You will understand the Jenkins architecture: master and agents, the executor model, and how to scale Jenkins for large organizations. You will learn Declarative Pipelines — the modern, structured way to define Jenkins pipelines as code — and how to use Shared Libraries to extract common pipeline logic into reusable functions that teams across an organization can use.

Modern Jenkins usage is tightly integrated with containers. You will learn how to run Jenkins agents dynamically in Kubernetes using the Kubernetes plugin, so that each build gets a fresh, isolated environment that scales automatically. This is the architecture used by large engineering organizations: a small, stable Jenkins master and ephemeral agent pods that spin up on demand and disappear after each build.

The module also covers the practical integration between Jenkins and the other tools in this course: triggering Terraform plans from Jenkins, building and scanning Docker images, deploying to Kubernetes, and integrating with GitHub for pull request feedback.

Daily Breakdown

Day 56: Jenkins Architecture, Installation, and Configuration as Code

Jenkins master and agent architecture. Installing Jenkins with Docker and as a Kubernetes deployment. Jenkins Configuration as Code (JCasC) with the casc plugin: defining Jenkins configuration in YAML files that can be stored in Git. Managing plugins programmatically. User authentication with LDAP/Active Directory. Role-based Authorization Strategy plugin for access control. Jenkins backup and disaster recovery.

Day 57: Declarative Pipelines, Shared Libraries, and Multi-branch Pipelines

Scripted vs Declarative Pipeline syntax and why Declarative is the modern standard. The full Declarative Pipeline syntax: agent, environment, options, parameters, stages, steps, post. Parallel stages for faster builds. Stash and unstash for passing artifacts between stages. Jenkins Shared Libraries: vars, src, and resources directories. Writing global pipeline functions. Multi-branch Pipeline for automatically creating jobs for every Git branch. Organization folders for GitHub/GitLab organization scanning.

Day 58: Dynamic Kubernetes Agents, Pipeline Security, and Integration Patterns

The Kubernetes plugin for Jenkins: podTemplates and container images as build environments. Running Maven, Node.js, Python, and Docker-in-Docker builds in Kubernetes pods. The Kaniko build approach for building Docker images without privileged containers. Pipeline security: credentials binding, secret management with HashiCorp Vault, and preventing secret leakage in logs. Integrating Jenkins with Jira, Slack, and email for notifications. Jenkins X as the cloud-native successor to Jenkins.

Hands-On Labs — Module 9

Hands-On 1: Set up a production-grade Jenkins installation on Kubernetes using Helm with Configuration as Code: configure RBAC, set up GitHub OAuth authentication, install and configure required plugins via JCasC, configure Kubernetes pod templates for Java, Node.js, and Python builds, and verify the setup is fully reproducible from the JCasC YAML files.

Hands-On 2: Build a complete Jenkins Declarative Pipeline for a microservices application: parallel build and test stages, Docker image build with Kaniko (no privileged mode), Trivy image scan with build failure on critical CVEs, push to ECR/ACR, deploy to Kubernetes dev namespace, integration tests, manual approval gate, and deploy to production with Slack notifications at every stage.

Hands-On 3: Create a Jenkins Shared Library with reusable functions for: standardized Docker image building and pushing, Kubernetes deployment with rollout verification, Slack notification formatting, and Trivy scanning with configurable thresholds. Update three existing pipelines to use the shared library and verify the pipelines still work correctly.

Hands-On 4: Configure a multi-branch pipeline strategy for a monorepo: automatic branch discovery, path-based triggers (only build service-a when service-a/ changes), PR validation builds, ephemeral review environments per pull request in Kubernetes, and automatic cleanup when a PR is closed.

Hands-On 5: Conduct a Jenkins security hardening exercise: enable audit logging, configure the Credentials Binding plugin to mask all secrets in console output, implement the OWASP dependency check plugin, set up the Job DSL plugin to prevent inline Groovy execution in pipelines, and run a security scan using the Jenkins security advisory checker plugin.



Module 10: Grafana, Prometheus, and Observability

You cannot manage what you cannot measure. In modern distributed systems, where a single user request might touch dozens of services across multiple data centers, understanding what is happening inside your system requires a sophisticated observability strategy. Observability is not just monitoring. Monitoring tells you when something is wrong. Observability tells you why.

The three pillars of observability are metrics, logs, and traces. Metrics are numerical measurements sampled over time: CPU usage, request latency, error rate, queue depth. Logs are timestamped records of discrete events: an HTTP request arrived, a database query executed, an error was thrown. Traces are the full journey of a request through a distributed system, showing which services it touched, how long each took, and where it failed.

Prometheus is the industry-standard metrics collection and storage system for cloud-native environments. It uses a pull model — scraping metrics endpoints on a configurable schedule — and a powerful query language called PromQL for extracting insights from time-series data. Prometheus is designed to work natively with Kubernetes: the kube-state-metrics and node-exporter integrations give you deep insight into cluster health out of the box.

Grafana is the visualization layer that transforms raw Prometheus data (and dozens of other data sources) into beautiful, actionable dashboards. Grafana's alerting engine evaluates PromQL expressions on a schedule and routes notifications to Slack, PagerDuty, email, and other channels when thresholds are breached. Understanding how to build dashboards that surface the right information at the right time is a critical engineering skill.

This module also introduces OpenTelemetry — the emerging standard for instrumentation that is gradually replacing vendor-specific SDKs — and Grafana Loki for log aggregation and Grafana Tempo for distributed tracing, completing the full observability stack.

Daily Breakdown

Day 59: Prometheus Architecture, PromQL, and the kube-prometheus Stack

Prometheus data model: time series, labels, metrics types (Counter, Gauge, Histogram, Summary). The scrape model and ServiceMonitor/PodMonitor resources. Installing the kube-prometheus stack with Helm: Prometheus Operator, Alertmanager, kube-state-metrics, node-exporter. PromQL fundamentals: selectors, range vectors, aggregation operators, functions. Writing recording rules and alerting rules. Alertmanager routing trees, inhibitions, and silences.

Day 60: Grafana Dashboards, Alerting, Loki, Tempo, and OpenTelemetry

Grafana architecture and data source configuration. Building dashboards from scratch: panels, variables, time range controls, and dashboard links. Using Grafonnet and Grafana dashboard-as-code approaches. Grafana Alerting: unified alerting, contact points, notification policies, and alert state management. Grafana Loki: log aggregation without indexing, LogQL query language, and log-to-metric derivation. Grafana Tempo for distributed tracing. OpenTelemetry Collector as the modern instrumentation pipeline. The future of observability: the OpenTelemetry standard and vendor-agnostic telemetry.

Hands-On Labs — Module 10

Hands-On 1: Deploy the kube-prometheus stack on your Kubernetes cluster, configure ServiceMonitor resources to scrape the microservices application you built earlier in the course, write PromQL queries to calculate the RED metrics (Rate, Errors, Duration) for each service, and create recording rules to pre-compute expensive queries.

Hands-On 2: Build a production-grade Grafana dashboard for your Kubernetes cluster: node resource utilization, pod restart rates, network I/O, persistent volume usage, and top consumers by namespace. Implement dashboard variables for filtering by namespace and node. Export the dashboard as JSON and store it in Git.

Hands-On 3: Configure a complete alerting pipeline: write Prometheus alerting rules for CPU saturation, memory pressure, pod crash-looping, and high error rate; configure Alertmanager to route critical alerts to PagerDuty and warnings to Slack; implement an inhibition rule to suppress node-level alerts when the cluster autoscaler is running; and verify alerts fire correctly by inducing the conditions.

Hands-On 4: Set up Grafana Loki for log aggregation: deploy Loki and Promtail on your Kubernetes cluster, configure Promtail to forward all pod logs, write LogQL queries to extract error rates and latency percentiles from structured logs, create a Grafana panel that links log entries to the corresponding Prometheus metrics for correlation.

Hands-On 5: Instrument a sample application with OpenTelemetry: add the OpenTelemetry SDK to a Node.js or Python service, configure the OTLP exporter to send traces to Grafana Tempo, configure metrics export to Prometheus, deploy the OpenTelemetry Collector as a Kubernetes DaemonSet, and build a unified Grafana dashboard that correlates metrics, logs, and traces for end-to-end request visibility.

Capstone Project: Build a Production-Ready Platform

The final week of the program is dedicated to a capstone project that integrates every technology you have learned. You will not be building a toy application. You will build a complete, production-grade platform that could actually run in a real company.

The capstone requires you to provision cloud infrastructure with Terraform from a blank account, configure a Kubernetes cluster with proper security and network policies, set up a CI/CD pipeline with Jenkins and GitHub Actions, deploy a microservices application with Docker containers, implement GitOps with ArgoCD for continuous deployment, configure comprehensive observability with Prometheus, Grafana, Loki, and Tempo, integrate security scanning at every pipeline stage, and document the entire architecture with runbooks for common operational tasks.

Capstone projects are reviewed by instructors and peers in a 30-minute demo session. The best capstone projects are featured on the program's website as portfolio examples. Every graduate receives a GitHub repository they own outright, with their entire infrastructure and application code, that serves as a compelling portfolio artifact for job applications.

What You Will Be Able to Do After 60 Days

After completing this program, you will not just understand DevOps concepts — you will have built the things that prove it. Here is what you will be capable of doing on your first week as a DevOps engineer at any organization, regardless of their specific tech stack.

You will be able to take a blank Linux server, harden it to production standards, install and configure any necessary runtime, write a monitoring agent configuration, and document everything as a runbook — all in under two hours. You will be able to containerize any application, write an optimized Dockerfile that produces a lean and secure image, push it to a registry, and deploy it to a Kubernetes cluster with proper resource limits, health checks, and auto-scaling configured.

You will be able to write Terraform code that provisions a complete cloud environment from scratch — VPC, compute, storage, networking, and security — with zero manual steps, in a way that can be reproduced identically for any new environment. You will be able to build CI/CD pipelines that take code from a developer's commit and deliver it to production automatically, with security scanning, automated testing, approval gates, and rollback capability.

You will be able to look at a Grafana dashboard for a system you have never seen before and, within minutes, identify which service is slow, which node is under pressure, and what changed 20 minutes ago that caused the problem. These are the skills that make DevOps engineers invaluable.

Visit our [website](#), fill out the enrollment form, and our admissions team will be in touch within 24 hours. The next cohort begins soon, and seats fill quickly. Do not wait.